1. **There exist two processes P_1 and P_2 in a multiprogramming batch system. P_2 enters into the system 10ms later than P_1, and their executing traces, i.e. alternating sequences of CPU bursts and I/O bursts, are as follows:**

P_1: computing, 80ms → I/O operation, 100ms → computing, 40ms

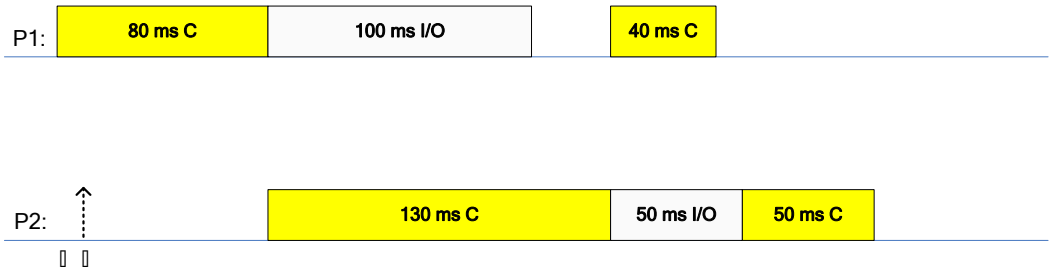
P_2: computing, 130ms → I/O operation, 50ms → computing, 50ms

It is assumed that time costs of CPU scheduling and process switch are omitted, what is the maximal throughput for completing these two processes, and why?

Answers：

1. 最大吞吐量 = 2/(80+130+50+50) = 2/310= **1/155** 个/ms

2. 当 **P_1、P_2** 的 CPU 计算和 I/O 操作最大程度并行执行时，2 个进程总的执行时间最短，系统吞吐量达到最大，如下图所示；



2. **In a computer system, the users submit to the system their computational tasks as jobs, and all these jobs are then stored as the standby jobs on the disk.**

The job scheduler (also known as *long-term scheduler*) selects the standby jobs on the disk, creates new processes in memory for them, and then starts executing of these processes. Each job's ID is the same as that of the process created for it, for example, J_i and P_i.

When the number of concurrent processes in memory is lower than *three*, the job scheduler takes the FCFS algorithm to select a standby job on the disk to create a new process. Otherwise, the jobs should wait on the disk.

For the processes in memory, the process scheduler (also known as *short-term scheduler*) takes the non-preemptive priority-based algorithm to select a process and allocates the CPU to it.

It is assumed the system costs resulting from job and process scheduling are omitted.

Consider the following set of Jobs J_1, J_2, J_3 , J_4 and J_5. For $1 \leq i \leq 5$, the arrival time of each J_i , the length of the CPU burst time of each process P_i, and the priority number for each J_i/P_i are given as below, and a smaller priority number implies a higher priority.

Job	Arrival Time	Burst Time (minute)	Priority Number
J_1	14:00	40	4
J_2	14:20	30.01	2
J_3	14:30	50.01	3
J_4	14:50	20.01	5
J_5	15:05	10.01	5

(1) Illustrate the execution of each job/process by charts.

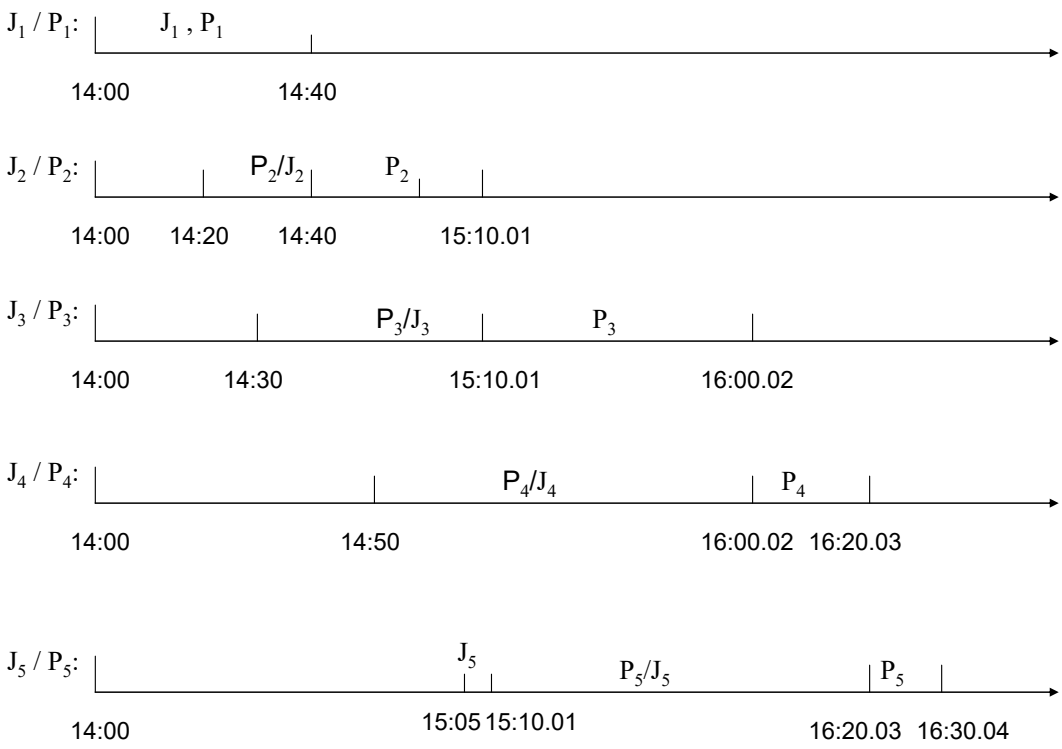
(2) What is the turnaround time of each job?

(3) What is the waiting time of each job?

Note: The waiting time of a job includes the time it waits on the disk and that it (as a process) waits in memory.

Answer:

（1）



注：图中 J_i 部分表示作业被调入内存，P_i 表示进程被调度执行。

J_1 到达时，内存中并发进程数=0<3，作业被直接调入内存，创建进程 P_1；P_1 被调度程序选中，开始执行。

在 P_1 执行过程中，J_2 到达，内存中并发进程数=1<3，作业被直接调入内存，创建进程 P_2；此时 P_1 在执行，由于采用非抢占式进程调度，P_2 处于就绪等待状态。

在 P_1 执行过程中，J_3 到达，内存中并发进程数=2<3，作业被直接调入内存，创建进程 P_3，等待 P_2 执行完毕。

P_1 结束后，作业 J_1 随之结束，系统内有作业 J_2、J_3 对对应的进程 P_2、P_3。进程调度选择高优先级的 P_2 开始执行。

P_2 执行过程中，J_4 到达时，内存中并发进程数=2<3，作业被直接调入内存，创建进程 P_4，等待 P_2 执行完毕。

P_2 执行过程中，J_5 到达时，内存中并发进程数=3，必须等到 P_2 结束后，系统内并发进程数<3，方能创建进程 P_5。

P_2 执行完毕后，系统内有 2 个作业 J_3、J_4 对应的进程 P_3、P_4，内存中并发进程数=2<3。此时，长期调度程序首先为作业 J_5 创建进程 P_5。然后，

进程调度程序从 P_3、P_4、P_5 中，选择高优先级的 P3 开始执行。

P_3 执行完毕后，系统内有 2 个作业 J_4、J_5 对应的进程 P_4、P_5，2 者的优先级相同，按照 FCFS 原则，P_4 先执行。

P_4 执行完毕后，剩余的唯一进程 P_5 开始执行。

（2）J_1 ： $T_1 = 40$ （min）

J_2 ： $T_2 = 20 + 30.01 \quad = 50.01$ （min）

J_3 ： $T_3 = 40.01 + 50.01 = 90.02$ （min）

J_4 ： $T_4 = 70.02 + 20.01 = 90.03$ （min）

J_5 ： $T_5 = 5.01 + 70.02 + 10.01 = 85.04$（min）

（3）J_1 ： $W_1 = 0$ （min）

J_2 ： $W_2 = 20$（min）

J_3 ： $W_3 = 40.01$（min）

J_4 ： $W_4 = 70.02$（min）

J_5 ： $W_5 = 75.03$（min）

3. Considering a real-time system, in which there are 4 real-time processes P_1, P_2, P_3 and P_4 that are aimed to react to 4 critical environmental events e_1, e_2, e_3 and e_4 in time respectively.

The arrival time of each event e_i , $1 \leqslant i \leqslant 4$, (that is, the arrival time of the process P_i), the length of the burst time of each process P_i, and the deadline for each event e_i are given below. Here, the deadline for e_i is defined as the absolute time point before which the process P_i must be completed.

The priority for each event e_i (also for P_i) is also given, and a smaller priority number implies a higher priority.

Events	Process	Arrival Time	Burst Time	Priorities	Deadline
e_1	P_1	0.00	4.00	3	7.00
e_2	P_2	3.00	2.00	1	5.50
e_3	P_3	4.00	2.00	4	12.01
e_4	P_4	6.00	4.00	2	11.00

(1) Suppose that priority-based preemptive scheduling is employed,

a) Draw a Gantt chart illustrating the execution of these processes

b) What are the average waiting time and the average turnaround time

c) Which event will be treated with in time ?

(2) Suppose that FCFS scheduling is employed,

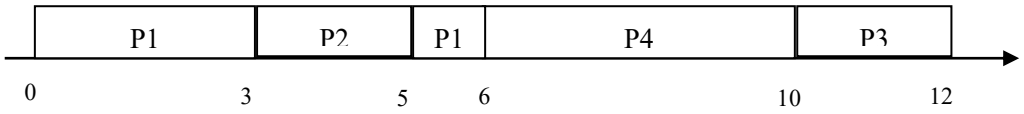
a) Draw a Gantt chart illustrating the execution of these processes

b) What are the average waiting time and the average turnaround time

c) Which event will be treated with in time, that is, the process reacting to this event will be completed before its deadline?

Answers:

(1) 甘特图如下

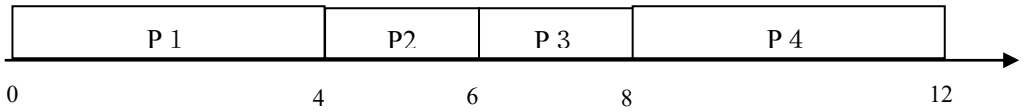


平均等待时间=[(5─3) + (3─3) + (10─4) + (6─6)] /4 = 2

平均周转时间=[(6─0) + (5─3) + (12─4) + (10─6)] /4 = 5

根据各进程的完成时间点和所对应的事件的 deadline 可知，全部 4 个事件均可得到及时响应.

(2) 甘特图如下

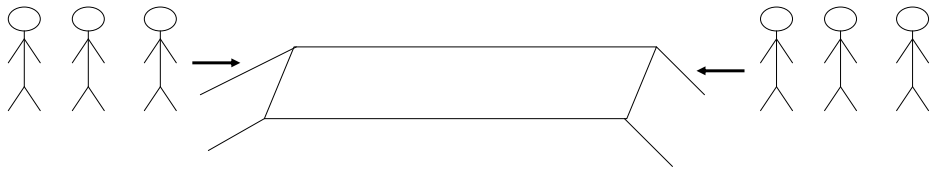


平均等待时间=[(0─0) + (4─3) + (6─4) + (8─6)] /4 = 1.25

平均周转时间=[(4─0) + (6─3) + (8─4) + (12─6)] /4 = 4.25

根据各进程的完成时间点和所对应的事件的 deadline 可知，事件 e1 和 e3 可得到及时响应.

4. **As illustrated in the figure, on** the **two sides of a one-plank bridge(独木桥), there are two groups of soldiers that are composed of *m* and *n* people respectively and need to cross the bridge, but the narrow bridge allows only one group of the soldiers in the same direction to cross at the same time. One group of the soldiers is permitted to cross as long as there are no people on the bridge. Once one group of the soldiers begins walking on the bridge, the other group should be waiting to start crossing until all members of the first group have passed the bridge.**



Please design two semaphore-based processes to describe the crossing actions of the soldiers in the two groups. It is required

(1) to define the semaphores and variables needed, explain their roles?, and give their initial values; and

(2) to illustrate the structures of processes for the soldiers in each group.

Answers:

该问题可归结为读写者问题，但 2 队士兵均为读者。

（1）定义 2 个整型变量 count1、count2，分别用于计数每组士兵过桥人数。

定义 2 个二元信号量 mutex1、mutex2，用于控制对 count1、count2 的互斥访问。

定义二元信号量 bridge，用于 2 队士兵间的互斥。

初始化：

count1=0，count2=0；

mutex1=1、mutex2=1；

bridge=1；

（2）

```
P1()
{
    wait(mutex1);
    count1 ++;
    if count1 = =1
      wait(bridge);   // 该组第 1 个士兵过桥时，阻塞另一组过桥
    signal(mutex1);
        ……
    crossing the bridge;
        ……
    wait(mutex1);
    count1 --;
    if count1 = =0
      signal(bridge);   // 该组最后 1 个士兵过桥后，允许另一组过桥
    signal(mutex1);
}.

P2()
{
    wait(mutex2);
    count2 ++;
    if count2 = =1
      wait(bridge);   // 该组第 1 个士兵过桥时，阻塞另一组过桥
    signal(mutex2);
        ……
    crossing the bridge;
```

```
        ……
    wait(mutex2);
    count2 --;
    if count2 = =0
       signal(bridge);   // 该组最后 1 个士兵过桥后，允许另一组过桥
    signal(mutex2);
}.
```

5. Consider the *bounded-buffer (also known as producer-consumer problem)*. There are several producer processes and consumer processes, and these processes share a pool of buffer. The pool of buffer consists of *n* buffers, and each buffer can hold only one item. The producer process produces one item and puts it into an empty buffer in the pool. The consumer process takes one item from a full buffer in the pool and consumes this item.

At each moment, *at most one producer process and at most one consumer process* are permitted to simultaneously operate on the buffer pool, i.e.,

- **one producer process is permitted to access the buffer lonely**
- **one consumer process is permitted to access the buffer lonely**
- **if two or more processes want to access the buffer simultaneously,**
  - **two or more *producer processes* are not permitted to simultaneously operate on the buffer, they should access the buffer mutual exclusively**
  - **two or more *consumer processes* are also not permitted to simultaneously operate on the buffer, they should access the buffer mutual exclusively**
  - **only one *producer process* and one *consumer process* are allowed to simultaneously operate on different items in the buffer**

Write an algorithm to synchronize producer processes and consumer processes by using semaphores.

Answer1:

(1) Principles:

define the semaphores ***empty*** and ***full*** to count the number of empty buffers and full buffers in the pool, respectively;

define binary semaphore ***mutex1*** to control all ***producers*** to access the empty buffers in the pool mutual exclusively;

define binary semaphore ***mutex2*** to control all ***consumers*** to access the full buffers in the pool mutual exclusively;

(2) Algorithm

Semaphores empty, full

Binary semaphores mutex1, mutex2

Initially,

empty:= n; full:=0; mutex1:=1; mutex2:=1;

The structure of the *producer* process is as follows:

```
do {    …
    produce an item in nextp
        …
    wait(empty);
    wait(mutex1);
    search for an empty buffer
        …
    add nextp to the empty buffer
        …
    signal(mutex1);
    signal(full);
} while (1);
```

The structure of the ***consumer*** process is as follows:

```
do {    …
    wait(full);
    wait(mutex2);
     search for an full buffer
       …
    remove item in buffer[j] to next ;
     ….
    signal(mutex2);
    signal(empty);
       …
    consume the item in nextc
       …
  } while (1);
```

Answer2:

Semaphores empty, full;

Binary semaphores mutex1, mutex2, mutex;

Array Flag[n]; /* Flag[i] =0 indicates that buffer[i] is empty,

Flag[i] =1 indicates that buffer[i] is full, 0≤ i ≤n-1;

*/

Initially,

empty:= n; full:=0; mutex1:=1; mutex2:=1; mutex=1;

Flag[i] =0, 0 ≤ i ≤n-1;

The structure of the *producer* process is as follows:

```
do {    …
    produce an item in nextp
      …
    wait(empty);
    wait(mutex1);

    wait(mutex);
    search on array Flag for an empty buffer[i], where Flag[i] =0;
    signal(mutex);
      …
    add nextp to the empty buffer[i];

    wait(mutex);
    Flag[i] : = 1;
    signal(mutex);

    signal(mutex1);
    signal(full);
     …
  } while (1);
```

The structure of the ***consumer*** process is as follows:

```
do {    …
    wait(full);
    wait(mutex2);

    wait(mutex);
    search on array Flag for an full buffer[j], where Flag[j] =1;
    signal(mutex);
      …
    remove item in buffer[j] to next ;

    wait(mutex);
    Flag[j] : = 0;
    signal(mutex);

    signal(mutex2);
    signal(empty);
      …
    consume the item in nextc
      …
  } while (1);
```

6. 有 1 个仓库，可以存放 X、Y 两种产品，仓库容量足够大，但要求：

1）每次只能存入一种产品 X 或 Y，

2）满足 —N < X 产品数量 — Y 产品数量 < M，M、N 为正整数。

用信号量实现产品 X、Y 的入库过程。

Answers：

产品数量制约条件：

X 产品数量 — Y 产品数量< M

X 产品数量 — Y 产品数量 > —N

设置 2 个信号量控制 X、Y 的数量：

1） sx 表示当前允许 X 产品比 Y 产品多入库的数量，即在当前库存量和 Y 产品不入库的情况下，还可以允许 sx 个 X 产品入库；初始时，若不放 Y 而仅放 X 产品，则 sx 最多为 M—1 个。

2） sy 表示当前允许 Y 产品比 X 产品多入库的数量，即在当前库存量和 X 产品不入库的情况下，还可以允许 sy 个 Y 产品入库；初始时，若不放 X 而仅放 Y 产品，则 sy 最多为 N—1 个。

当往库中放入一个 X 产品时，则允许存入 Y 产品的数量加 1，故信号量 sy 应加 1；

当往库中放入一个 Y 产品时，则允许存入 X 产品的数量加 1，故信号量 sx 应加 1；

Mutex ： semaphore=1 互斥信号量

sx, sy: semaphore

sx=M-1, sy=N-1

Process X

Repeat

Wait(sx)

Wait(mutex)

将 X 入库

Signal(mutex)

Signal(sy)

Until false

Process Y

Repeat

Wait(sy)

Wait(mutex)

将 Y 入库

Signal(mutex)

Signal(sx)

Until false

7. Here are three workers W_1, W_2 and W_3. W_1 and W_2 are responsible for producing the part(零件) P_1 and part P_2 respectively, and W_3 takes charge of assembling (装配) the part P_3.

W_1 produces one P_1 and then puts it into the cargo tank(货箱) T_1; W_2 manufactures one P_2 and then puts it into the cargo tank T_2. W_3 takes P_1 from T_1 and P_2 from T_2, and then assembles P_1 and P_2 into P_3.

It is assumed that (1) T_1 can hold 8 parts of P_1, and T_2 can hold 10 parts of P_2, and they are all initially empty; (2) each time W_3 can takes only one part of P_1 from T_1 and one part of P_2 from T_2, and then assembles one part of P_3; (3) T_1 and T_2 are permitted only to be taken or put in the mutually exclusive mode, that is, at any time only one worker are allowed to operate on T_1 or T_2.

Please design three semaphore-based programs for W_1, W_2 and W_3, to describe their production activities. It is required

(1) to define the semaphores for synchronizing the three processes, explain the role of each semaphore, and give their initial values; and

(2) to illustrate the structures of processes for W_1, W_2 and W_3.

Answers:

(1)

信号量定义：

Binary semaphore mutex1, mutex2

Semaphore empty1, full1, empty2, full2

信号量含义：

mutex1 用于控制 W_1 和 W_2 对 T1 的互斥访问，mutex2 用于控制 W_1 和 W3 对 T_2 的互斥访问；

empty1 和 empty2 分别表示 T_1 和 T_2 中的空闲容量，full1 和 full2 分别表示 T_1 和 T_2 中已经被占用的容量。

信号量初值：

mutex1=1; mutex2=1;

empty1=8; empty2=10;

full1=0; full2=0;

（2）

W_1：

Produce one P_1 ;

Wait(empty1);

Wait(mutex1);

Puts it into T_1,

Signal(mutex1);

Signal(full1);

W_2：

Produce one P_2 ;

Wait(empty2);

Wait(mutex2);

Puts it into T_2,

Signal(mutex2);

Signal(full2);

W_3:

Wait(full1);
Wait(mutex1);
takes one P_1 from T_1;
Signal(mutex1);
Signal(empty1);

Wait(full2);
Wait(mutex2);
takes one P_2 from T_2;
Signal(mutex2);
Signal(empty2);

Assembles P_1 and P_2 into P_3.

8. In a data collection system, there are a data collection process, a data manipulation process, and a data output process. The Data collection process puts one unit of data collected into Buffer 1, the data manipulation process gets one unit of data from Buffer 1 to manipulate and puts the results into Buffer 2, and then the data output process gets one unit of data from Buffer 2, and outputs them. It is assumed that Buffer 1 can keep three units of data and Buffer 2 can keep two units of data at most.

1) Define the semaphores used to synchronize the three processes, and set their initial values.

2) Please design three processes for the data collection process, the data manipulation process, and the data output process respectively by using semaphores defined in 1).

Answers:

(1) (4 points)

binary semaphore mutex1, mutex2

semaphore full1, full2, empty1, empty2

mutex1=1; mutex2=1;

empty1=3; full1=0; empty2=2; full2=0;

(2) (4×3=12 poin ts)

a. the collection process:

```
collect one unit of data;
wait(empty1);
wait(mutex1);
  put the data into buffer1;
signal(mutex1);
signal(full1);
```

b. the manipulating process:

```
wait(full1);
wait(mutex1);
   remove one unit of data from buffer1;
signal(mutex1);
signal(empty1);
manipulate the data;
wait(empty2);
wait(mutex2);
  put the data into buffer2;
signal(mutex2);
signal(full2);
```

c. the output process:

```
wait(full2);
wait(mutex2);
   remove one unit of data from buffer2;
signal(mutex2);
signal(empty2);
output the data;
```